In [1]:

```python
import os
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

1)

Write a program to display an image and the modulus of its Fourier transform in 2D.

- What does the FFT 2D calculation function return?
- By default, what is the size of the outputtable?
- Can this size be changed?
- What is the fftshift function used for ?
- Locate high and low frequencies.

In [10]:

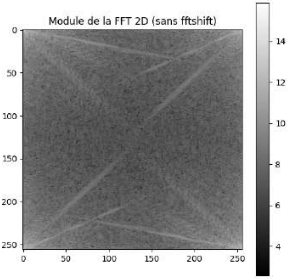
```python
# Nom de l'image
img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE)


# Convertir l'image en tableau NumPy
I = np.array(img, dtype=np.float32)


print("Taille de l'image :", I.shape)


plt.figure(figsize=(6, 6))

plt.imshow(I, cmap="gray")
plt.title("Image originale")
plt.axis("off")

plt.show()


#  Calcul de la transformée de Fourier 2D

F = np.fft.fft2(I)

# Afficher la taille de la FFT
print("Taille de la FFT 2D :", F.shape)

# Afficher quelques informations
print("Type des données de la FFT :", F.dtype)


# Calcul du module de la transformée de Fourier
M = np.abs(F)


# Afficher le module de la FFT sans fftshift

plt.figure(figsize=(6, 6))

plt.imshow(np.log(1 + M), cmap="gray")
plt.title("Module de la FFT 2D (sans fftshift)")
plt.colorbar()

plt.show()


# Utilisation de fftshift
F_shift = np.fft.fftshift(F)

# Calcul du module après fftshift
M_shift = np.abs(F_shift)


#  Afficher le spectre centré


plt.figure(figsize=(7, 7))

plt.imshow(np.log(1 + M_shift), cmap="gray")

plt.title("Module de la FFT 2D avec fftshift")
plt.xlabel("Fréquence horizontale")
plt.ylabel("Fréquence verticale")

plt.colorbar(label="Magnitude logarithmique")

plt.show()


# 8. Afficher l'image et son spectre côte à côte

plt.figure(figsize=(12, 5))

# Image originale
plt.subplot(1, 2, 1)
plt.imshow(I, cmap="gray")
plt.title("Image originale")
plt.axis("off")

# Spectre de Fourier
plt.subplot(1, 2, 2)
plt.imshow(np.log(1 + M_shift), cmap="gray")
plt.title("Module de la FFT 2D")
plt.xlabel("Fréquence horizontale")
plt.ylabel("Fréquence verticale")
plt.colorbar()
```

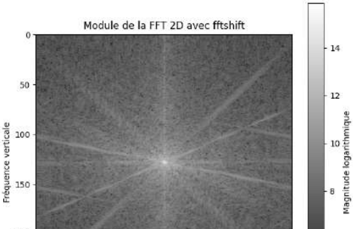
```python
plt.tight_layout()
plt.show()


# Modifier la taille de la FFT


# Exemple : calculer une FFT de taille 512 x 512
F_512 = np.fft.fft2(I, s=(512, 512))

print("Taille de la FFT avec s=(512,512) :", F_512.shape)

# Localisation des basses et hautes fréquences


plt.figure(figsize=(7, 7))

plt.imshow(
    np.log(1 + np.abs(np.fft.fftshift(F))),
    cmap="gray"
)

plt.title("Localisation des fréquences")
plt.xlabel("← Hautes fréquences       Basses fréquences       Hautes fréquences →")
plt.ylabel("← Hautes fréquences\n\nBasses fréquences\n\nHautes fréquences →")

plt.colorbar(label="Magnitude")

plt.show()
```

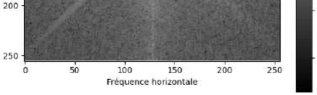
```
Taille de l'image : (256, 256)
```

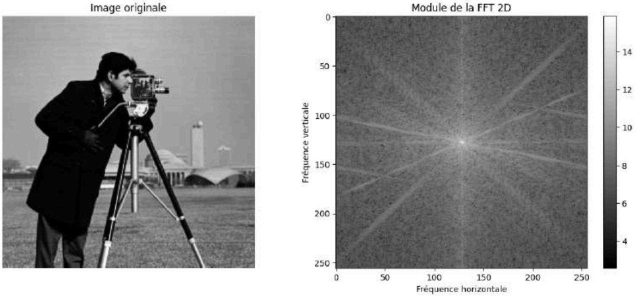
Image originale

```
Taille de la FFT 2D : (256, 256)
Type des données de la FFT : complex64
```

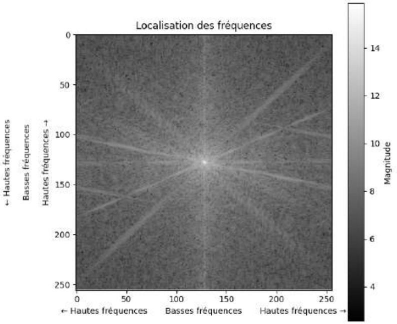








```
Taille de la FFT avec s=(512,512) : (512, 512)
```



RÉSUMÉ

np.fft.fft2(I) calcule la transformée de Fourier 2D.

Le résultat est un tableau complexe.

Par défaut, la FFT possède la même taille que l'image.

La taille peut être modifiée avec s=(M,N).

np.abs(F) donne le module de la FFT.

np.fft.fftshift(F) place les basses fréquences au centre.

Localisation des hautes et bassses fréquences

Centre du spectre = basses fréquences.

Bords du spectre = hautes fréquences.

2)

Observe the modulus of the 2D Fourier transform of several grayscale images: cameraman.gif, bureau.gif, testpat1.tif, testpat2.tif, circuit.tif, ic.tif, ic45.tif -Why is it better to view the TF module log?

-What does an image rotation on the TF produce?

-What does the TF(0,0) element represent?

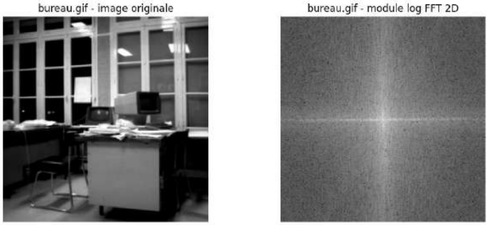
-What is the relationship between this element, the average and the size of the image?

In [12]:

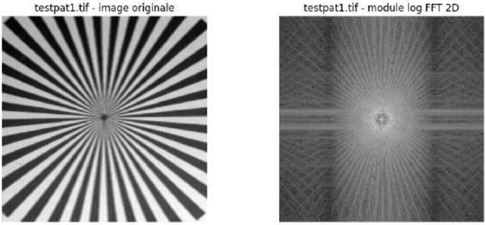
```python
# Observation du module de la transformée de Fourier 2D pour plusieurs images en niveaux de gris
import os
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

```python
image_names = [
    'cameraman.gif',
    'bureau.gif',
    'testpat1.tif',
    'testpat2.tif',
    'circuit.tif',
    'ic.tif',
    'ic45.tif'
]

# Chercher les images dans différents dossiers possibles
search_dirs = ['.', 'Images', 'images']

def find_image(path_name):
    for folder in search_dirs:
        candidate = os.path.join(folder, path_name)
        if os.path.exists(candidate):
            return candidate

    for root, _, files in os.walk('.'):
        if path_name in files:
            return os.path.join(root, path_name)

    raise FileNotFoundError(f"Image introuvable : {path_name}")

for name in image_names:
    try:
        image_path = find_image(name)
        img = cv.imread(image_path, cv.IMREAD_GRAYSCALE)

        if img is None:
            print(f"Impossible de lire : {name}")
            continue

        I = img.astype(np.float32) # Convertir en float32 pour la FFT
        F = np.fft.fft2(I)  # Calculer la FFT 2D
        F_shift = np.fft.fftshift(F)   # Centrer les basses fréquences
        M = np.log(1 + np.abs(F_shift)) # Calculer le module de la FFT 2D

        plt.figure(figsize=(10, 4))

        plt.subplot(1, 2, 1)
        plt.imshow(I, cmap='gray')
        plt.title(f'{name} - image originale')
        plt.axis('off')

        plt.subplot(1, 2, 2)
        plt.imshow(M, cmap='gray')
        plt.title(f'{name} - module log FFT 2D')
        plt.axis('off')

        plt.tight_layout()
        plt.show()

        print(f"Image : {name}")
        print("Taille image :", I.shape)
        print("Taille FFT :", F.shape)
        print("Valeur du pixel central FFT(0,0) :", F_shift[0, 0])
        print("Moyenne de l'image :", I.mean())
        print("---")

    except FileNotFoundError as e:
        print(e)

print("Fin de l'observation des images.")
```

```
Image introuvable : cameraman.gif
```

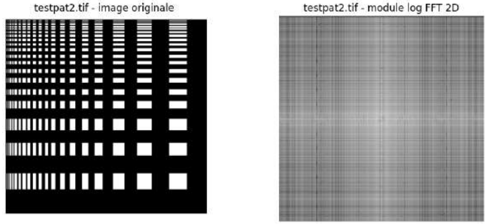


```
Image : bureau.gif
Taille image : (256, 256)
Taille FFT : (256, 256)
Valeur du pixel central FFT(0,0) : (424+0j)
Moyenne de l'image : 80.44647
---
```


testpat1.tif - image originale

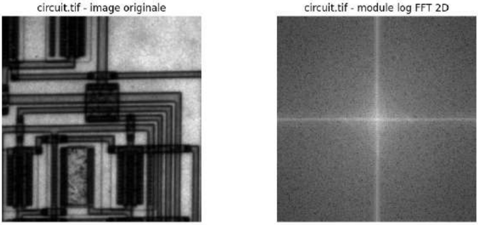
testpat1.tif - module log FFT 2D

```
Image : testpat1.tif
Taille image : (256, 256)
Taille FFT : (256, 256)
Valeur du pixel central FFT(0,0) : (-161+0j)
Moyenne de l'image : 124.69673
---
```


testpat2.tif - image originale

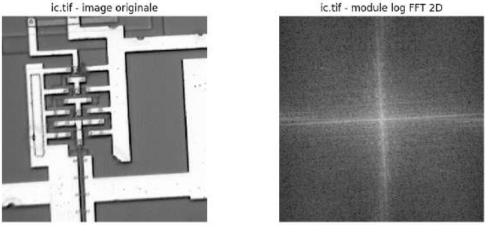
testpat2.tif - module log FFT 2D

```
Image : testpat2.tif
Taille image : (256, 256)
Taille FFT : (256, 256)
Valeur du pixel central FFT(0,0) : 0j
Moyenne de l'image : 69.92111
---
```


circuit.tif - image originale

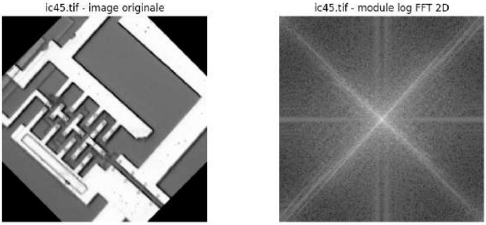
circuit.tif - module log FFT 2D

```
Image : circuit.tif
Taille image : (280, 272)
Taille FFT : (280, 272)
Valeur du pixel central FFT(0,0) : (-69+1.7053026e-13j)
Moyenne de l'image : 76.25823
---
```


ic.tif - image originale

ic.tif - module log FFT 2D

```
Image : ic.tif
Taille image : (256, 256)
Taille FFT : (256, 256)
Valeur du pixel central FFT(0,0) : (208+0j)
Moyenne de l'image : 155.2001
---
```



```
Image : ic45.tif
Taille image : (256, 256)
Taille FFT : (256, 256)
Valeur du pixel central FFT(0,0) : (-1858+0j)
Moyenne de l'image : 130.28531
---
Fin de l'observation des images.
```

Interpretation

Il vaut-il mieux regarder le log du module de la TF Parce que le spectre de Fourier a une très grande dynamique de valeurs. Le terme central $F(0,0)$ est souvent beaucoup plus grand que les autres coefficients, ce qui masque les détails faibles. En utilisant le logarithme du module :

- on compresse la plage de valeurs,
- on rend visibles les basses amplitudes,
- on peut mieux observer les structures et les variations de fréquence.

'

Une rotation de l'image dans le domaine spatial produit la même rotation dans le domaine de Fourier. Si l'image est tournée d'un angle θ, alors son spectre est aussi tourné de θ. Autrement dit, la transformation de Fourier conserve l'orientation.

'

l'élément $TF(0,0)$ représente le composant DC, c'est-à-dire la fréquence nulle. Il correspond à la moyenne globale de l'intensité de l'image.

Mathématiquement :

$$F(0,0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x,y)$$

Relation entre $F(0,0)$, la moyenne et la taille de l'image

Si l'image a une taille $M \times N$, alors :

$$\text{moyenne}(I) = \frac{F(0,0)}{M \times N}$$

Donc :

- $F(0,0)$ est la somme de tous les pixels,
- la moyenne est cette somme divisée par le nombre total de pixels,
- plus l'image est grande, plus $F(0,0)$ est grand, mais la moyenne reste la même si l'intensité moyenne est identique.

Formule clé :

$$F(0,0) = M \times N \times \text{moyenne}(I)$$

3)

By applying a circular mask to the TF, remove some of the frequencies (low, high or medium), then apply the reverse TF to observe the changes in the image. Test several images: lena.bmp, barbara.gif, testpat1.tif...

Intuitively, where are the high and low frequencies located in an image?

<VSCode.Cell language="markdown">

Résumé rapide

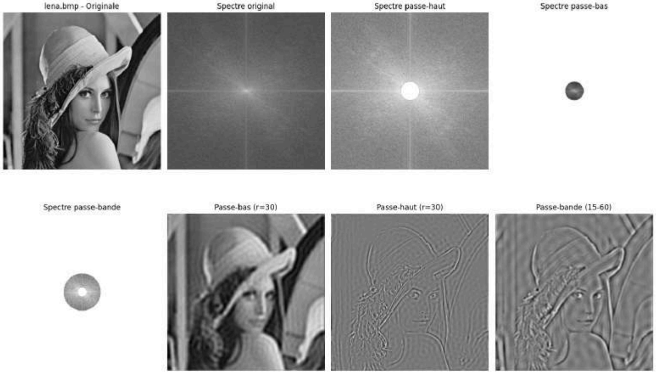
- `FFT` : transforme une image du domaine spatial vers le domaine fréquentiel.
- `FFT_SHIFT` : décale le spectre pour placer les basses fréquences au centre.
- `MASK` : masque le spectre pour garder ou éliminer certaines fréquences.
- `FILTRE` :
  - passe-bas = garde les basses fréquences, enlève les détails fins,
  - passe-haut = garde les contours, enlève les zones homogènes.
- `IFFT` : retransforme le spectre filtré dans le domaine spatial.
- `np.log(1 + np.abs(F))` : affiche le module du spectre avec un rendu plus lisible, car les amplitudes varient beaucoup.

In [23]:

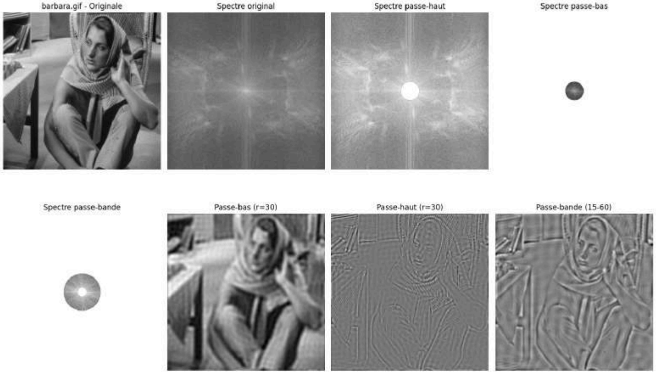
```python
# Liste des images à tester
image_names = ['lena.bmp', 'barbara.gif', 'testpat1.tif']

# Recherche des images dans plusieurs dossiers possibles
search_dirs = ['images']

def find_image(path_name):
    for folder in search_dirs:
        candidate = os.path.join(folder, path_name) # Construire le chemin de l'image
        if os.path.exists(candidate):
            return candidate
```

```python
    for root, _, files in os.walk('.'):
        if path_name in files:
            return os.path.join(root, path_name)

    raise FileNotFoundError(f"Image introuvable : {path_name}")


def circular_mask(shape, radius, kind='low'):
    rows, cols = shape # Obtenir les dimensions de la masque
    cy, cx = rows // 2, cols // 2 # Calculer le centre du masque
    Y, X = np.ogrid[:rows, :cols] # Créer une grille de coordonnées pour le masque
    distance = np.sqrt((X - cx) ** 2 + (Y - cy) ** 2) # Calculer la distance de chaque pixel par rapport au centre

    if kind == 'low':
        return (distance <= radius).astype(np.float32) # Masque passe-bas
    elif kind == 'high':
        return (distance > radius).astype(np.float32) # Masque passe-haut
    elif kind == 'band':
        return ((distance >= radius[0]) & (distance <= radius[1])).astype(np.float32) # Masque passe-bande
    else:
        raise ValueError("kind doit être 'low', 'high' ou 'band'")


for name in image_names:
    try:
        img_path = find_image(name)
        img = cv.imread(img_path, cv.IMREAD_GRAYSCALE)

        if img is None:
            print(f"Erreur de lecture : {name}")
            continue

        # FFT 2D
        F = np.fft.fft2(img) # Calculer la FFT 2D
        F_shift = np.fft.fftshift(F) # Centrer les basses fréquences

        # Masques circulaires
        low_mask = circular_mask(F_shift.shape, radius=30, kind='low') # Masque passe-bas
        high_mask = circular_mask(F_shift.shape, radius=30, kind='high')
        band_mask = circular_mask(F_shift.shape, radius=(15, 60), kind='band')

        # Filtrage
        F_low = F_shift * low_mask
        F_high = F_shift * high_mask
        F_band = F_shift * band_mask

        # FFT inverse
        img_low = np.fft.ifft2(np.fft.ifftshift(F_low)).real # Filtrage passe-bas
        img_high = np.fft.ifft2(np.fft.ifftshift(F_high)).real # Filtrage passe-haut
        img_band = np.fft.ifft2(np.fft.ifftshift(F_band)).real # Filtrage passe-bande

        spectre_low = 20*np.log(np.abs(F_low))
        spectre_high = 20*np.log(np.abs(F_high))
        spectrre_band = 20*np.log(np.abs(F_band))
        spectre_original = 20*np.log(np.abs(F_shift))

        # Affichage
        plt.figure(figsize=(15, 10))

        plt.subplot(2, 4, 1)
        plt.imshow(img, cmap='gray')
        plt.title(f'{name} - Originale')
        plt.axis('off')

        plt.subplot(2, 4, 2)
        plt.imshow(spectre_original, cmap='gray')
        plt.title('Spectre original')
        plt.axis('off')



        plt.subplot(2, 4, 3)
        plt.imshow(spectre_high, cmap='gray')
        plt.title('Spectre passe-haut')
        plt.axis('off')

        plt.subplot(2, 4, 4)
        plt.imshow(spectre_low, cmap='gray')
        plt.title('Spectre passe-bas')
        plt.axis('off')

        plt.subplot(2, 4, 5)
        plt.imshow(spectrre_band, cmap='gray')
        plt.title('Spectre passe-bande')
        plt.axis('off')


        plt.subplot(2, 4, 6)
        plt.imshow(img_low, cmap='gray')
        plt.title('Passe-bas (r=30)')
        plt.axis('off')

        plt.subplot(2, 4, 7)
        plt.imshow(img_high, cmap='gray')
        plt.title('Passe-haut (r=30)')
        plt.axis('off')

        plt.subplot(2, 4, 8)
        plt.imshow(img_band, cmap='gray')
        plt.title('Passe-bande (15-60)')
        plt.axis('off')

        plt.tight_layout()
        plt.show()

        print(f"Image testée : {name}")
        print("Les basses fréquences sont au centre du spectre.")
        print("Les hautes fréquences sont aux bords du spectre.")
        print("---------------------------------------------------")

    except FileNotFoundError as e:
        print(e)

print("Fin du test des filtres fréquentiels.")
```

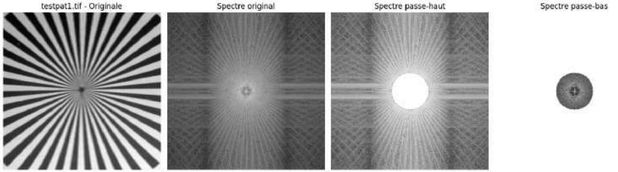
```
/var/folders/qr/5mnvs57d30q1fn3bmgkp39kc0000gn/T/ipykernel_83189/296016871.py:64: RuntimeWarning: divide by zero encountered in log
  spectre_low = 20*np.log(np.abs(F_low))
/var/folders/qr/5mnvs57d30q1fn3bmgkp39kc0000gn/T/ipykernel_83189/296016871.py:65: RuntimeWarning: divide by zero encountered in log
  spectre_high = 20*np.log(np.abs(F_high))
/var/folders/qr/5mnvs57d30q1fn3bmgkp39kc0000gn/T/ipykernel_83189/296016871.py:66: RuntimeWarning: divide by zero encountered in log
  spectrre_band = 20*np.log(np.abs(F_band))
```

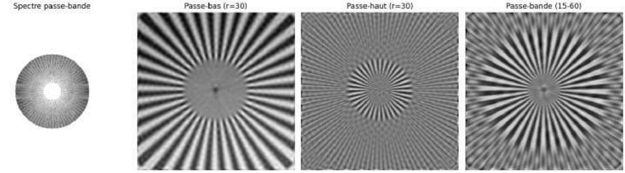


```
Image testée : lena.bmp
Les basses fréquences sont au centre du spectre.
Les hautes fréquences sont aux bords du spectre.
--------------------------------------------------
```



```
Image testée : barbara.gif
Les basses fréquences sont au centre du spectre.
Les hautes fréquences sont aux bords du spectre.
--------------------------------------------------
```





```
Image testée : testpat1.tif
Les basses fréquences sont au centre du spectre.
Les hautes fréquences sont aux bords du spectre.
--------------------------------------------------
Fin du test des filtres fréquentiels.
```

Résultat

En appliquant un masque circulaire sur le spectre de Fourier, on peut enlever certaines fréquences (basses, hautes ou intermédiaires), puis reconstruire l'image avec la transformée de Fourier inverse pour observer l'effet.

Explication

- Les basses fréquences correspondent aux variations lentes et aux régions homogènes de l'image.
- Les hautes fréquences correspondent aux contours, textures, détails fins et transitions brusques.
- Dans le spectre de Fourier, les basses fréquences sont près du centre.
- Les hautes fréquences sont loin du centre.

Donc :

- un filtre passe-bas (low-pass) garde le centre et supprime les détails fins,
- un filtre passe-haut (high-pass) garde les bords et supprime les zones uniformes,
- un filtre passe-bande garde uniquement une plage de fréquences.

En pratique :

$$F(u,v) = \mathcal{F}\{I(x,y)\}$$

et la reconstruction se fait par :

$$I'(x,y) = \mathcal{F}^{-1}\{F(u,v) \cdot M(u,v)\}$$

où $M(u,v)$ est le masque appliqué au spectre.

Donc

- Les basses fréquences dans le spectre se trouveent au centre.
- Les hautes fréquences dans le spectre se trouveent aux bords du spectre.
- Si on enlève les hautes fréquences l'image devient plus floue, avec moins de détails.
- Si on enlève les basses fréquences, les contours et les détails restent, mais l'image perd son aspect global et devient plus "filigrane".

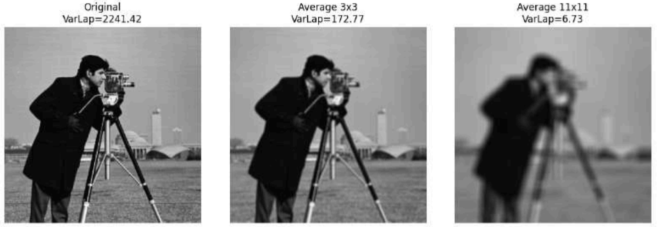
4)

To start, apply a 3x3 size averaging filter on the image: Start again with 11x11 size averaging filter. Evaluate the sharpness of the image with a quantitative indicator, such as the variance of the Laplacian, for example. What effect do they have? What is the size of the output image? Evaluate the side effects.

In [ ]:

```python
# Use existing `img` if present, otherwise try to load from img_path

img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError(f"Cannot load image from {img_path}")

# Averaging (box) filters
blur3 = cv.blur(img, (3, 3))
blur11 = cv.blur(img, (11, 11))

# Sharpness metric: variance of the Laplacian
def laplacian_variance(im):
    L = cv.Laplacian(im, cv.CV_64F)
    return float(L.var())

v_orig = laplacian_variance(img)
v_3 = laplacian_variance(blur3)
v_11 = laplacian_variance(blur11)

# Print quantitative results and output sizes
print(f"Original: shape={img.shape}, Laplacian variance={v_orig:.2f}")
print(f"Blur 3x3: shape={blur3.shape}, Laplacian variance={v_3:.2f}")
print(f"Blur 11x11: shape={blur11.shape}, Laplacian variance={v_11:.2f}")

# Display images side-by-side with metric in the title
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray', vmin=0, vmax=255)
plt.title(f'Original\nVarLap={v_orig:.2f}')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(blur3, cmap='gray', vmin=0, vmax=255)
plt.title(f'Average 3x3\nVarLap={v_3:.2f}')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(blur11, cmap='gray', vmin=0, vmax=255)
plt.title(f'Average 11x11\nVarLap={v_11:.2f}')
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Original: shape=(256, 256), Laplacian variance=2241.42
Blur 3x3: shape=(256, 256), Laplacian variance=172.77
Blur 11x11: shape=(256, 256), Laplacian variance=6.73
```



Interprétation :

-La variance du Laplacien est une mesure de netteté. Une valeur plus élevée indique plus de détails et de bords dans l'image.

-À mesure que nous appliquons des filtres de moyenne plus grands, l'image devient plus floue et la variance du Laplacien diminue, indiquant une perte de netteté.

-la taille de l’image de sortie reste la même que la taille de l’image d’entrée pour les deux opérations de flou. Les effets secondaires de l’application de filtres de moyenne incluent une réduction de la netteté et des détails de l’image, comme en témoigne la diminution de la métrique de variance laplacienne. Plus la taille du noyau est grande (par exemple 11x11), plus l'effet de flou est prononcé, conduisant à une image plus fluide avec moins de contenu haute fréquence.

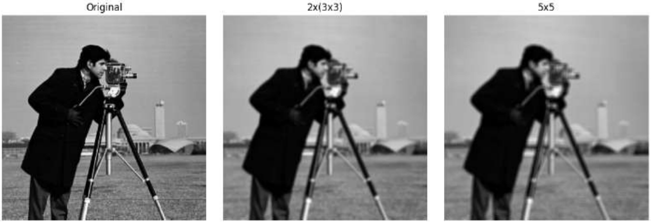
5)

Compare the result obtained by filtering the original image twice in a row with the 3x3 averaging (re-applying the convolution on the filtered image obtained), and the result obtained by filtering the original image with a 5x5 averaging device. We can realize a difference between the two resulting images. What is the best option in terms of calculations? What advantages does local processing bring compared to a global transformation such as the Fourier transform?

In [37]:

```python
import time
import numpy as np
import cv2 as cv

import matplotlib.pyplot as plt

# use existing `img` and `laplacian_variance` if available, otherwise provide fallback
try:
    laplacian_variance
except NameError:
    def laplacian_variance(im):
        L = cv.Laplacian(im, cv.CV_64F)
        return float(L.var())

# two successive 3x3 average filters
t0 = time.perf_counter()
kernel = np.ones((3,3), dtype=np.float32) / 9
r_3x3_twice = cv.filter2D(img, -1, kernel)
r_3x3_twice = cv.filter2D(r_3x3_twice, -1, kernel)
t1 = time.perf_counter()



# single 5x5 average filter
t2 = time.perf_counter()
r_5x5 = cv.filter2D(img, -1, np.ones((5,5), dtype=np.float32) / 25)
t3 = time.perf_counter()

# metrics
time_3x3_twice = t1 - t0
time_5x5 = t3 - t2
var_3x3_twice = laplacian_variance(r_3x3_twice)
var_5x5 = laplacian_variance(r_5x5)
mse_between = np.mean((r_3x3_twice.astype(np.float32) - r_5x5.astype(np.float32)) ** 2)

print(f"2x(3x3): time={time_3x3_twice:.6f}s  LapVar={var_3x3_twice:.2f}")
print(f"5x5    : time={time_5x5:.6f}s      LapVar={var_5x5:.2f}")
print(f"MSE between results: {mse_between:.4f}")

# visual comparison
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap='gray'); plt.title('Original'); plt.axis('off')
plt.subplot(1, 3, 2); plt.imshow(r_3x3_twice, cmap='gray'); plt.title('2x(3x3)'); plt.axis('off')
plt.subplot(1, 3, 3); plt.imshow(r_5x5, cmap='gray'); plt.title('5x5'); plt.axis('off')
plt.tight_layout()
plt.show()

# concise conclusion
if time_3x3_twice < time_5x5:
    print("Cheapest: two 3x3 convolutions (lower total kernel area => fewer operations).")
else:
    print("Cheapest: single 5x5 convolution.")
```

```
2x(3x3): time=0.035305s  LapVar=58.41
5x5    : time=0.000414s      LapVar=46.17
MSE between results: 16.3823
```



```
Cheapest: single 5x5 convolution.
```

Interprettattion:

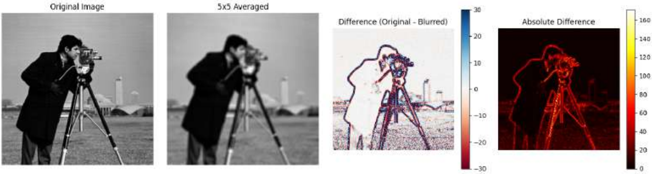
les résultats diffèrent, deux 3x3 donnent souvent un profil de lissage légèrement différent ; le traitement local est moins cher pour les petits noyaux, simple à appliquer de manière adaptative et préserve la localité, tandis que les méthodes de Fourier/globales sont préférables pour les grandes convolutions ou la conception explicite dans le domaine fréquentiel

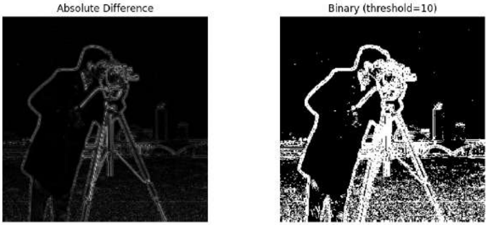
6)

Subtract the image filtered by a 5x5 averaging from the original image. Visualize the result obtained, then its absolute value. Explain what this result represents. Binarize the image obtained by thresholding and display the binary image.

In [ ]:

```
# Subtract the 5x5 averaged image from the original
diff = img.astype(np.float32) - r_5x5.astype(np.float32) # c'est la différence entre l'image originale et l'image floutée

# Absolute value of the difference
abs_diff = np.abs(diff)

# Binarize by thresholding (e.g., threshold = 10)
threshold = 10
binary_img = (abs_diff > threshold).astype(np.uint8) * 255. # cest-à-dire que si la différence absolue est supérieure au seuil, on met 255 (b

# Display
plt.figure(figsize=(15, 4))

plt.subplot(1, 4, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(r_5x5, cmap='gray')
plt.title('5x5 Averaged')
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(diff, cmap='RdBu', vmin=-30, vmax=30)
plt.title('Difference (Original - Blurred)')
plt.colorbar()
plt.axis('off')

plt.subplot(1, 4, 4)
plt.imshow(abs_diff, cmap='hot')
plt.title('Absolute Difference')
plt.colorbar()
plt.axis('off')

plt.tight_layout()
plt.show()


# Display binary result
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(abs_diff, cmap='gray')
plt.title('Absolute Difference')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(binary_img, cmap='gray')
plt.title(f'Binary (threshold={threshold})')
plt.axis('off')

plt.tight_layout()
plt.show()

print(f"Difference range: [{diff.min():.2f}, {diff.max():.2f}]")
print(f"Absolute difference range: [{abs_diff.min():.2f}, {abs_diff.max():.2f}]")
print(f"Binary image: {binary_img.sum()} pixels above threshold")

# Conclusion: The difference image highlights areas of the original image that are significantly altered by the 5x5 averaging filter. The bin
```





```
Difference range: [-152.00, 171.00]
Absolute difference range: [0.00, 171.00]
Binary image: 3923685.0 pixels above threshold
```

Interpretation:

le résultat obtenu en appliquant une moyenne 5x5 sur la différence représente les zones de l'image originale qui sont significativement altérées par le filtre de moyenne 5x5, mettant en évidence les bords et le contenu haute fréquence.

L'image binaire segmente efficacement ces zones en fonction du seuil spécifié, permettant une identification facile des régions avec un contenu ou des bords haute fréquence.

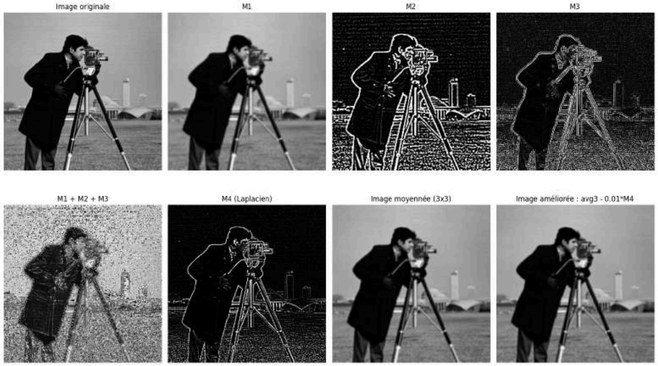
7)

Try the following masks on the images at your disposal and deduce the filtering effect produced: low-pass, band-pass, high-pass, ...

In [46]:

```python
# Charger une image en niveaux de gris
img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image introuvable")

# --- Masques donnés ---
M1 = np.array([
    [1, 4, 7, 4, 1],
    [4, 16, 26, 16, 4],
    [7, 26, 41, 26, 7],
    [4, 16, 26, 16, 4],
    [1, 4, 7, 4, 1]
], dtype=np.float32) / 256.0

M2 = np.array([
    [-0.61, -2, -2.85, -2, -0.61],
    [-2, 1.61, 3.85, 1.61, -2],
    [-2.85, 3.85, 8, 3.85, -2.85],
    [-2, 1.61, 3.85, 1.61, -2],
    [-0.61, -2, -2.85, -2, -0.61]
], dtype=np.float32)

M3 = np.array([
    [-0.38, 1, 1.85, 1, -0.38],
    [1, -2.61, -4.85, -2.61, 1],
    [1.85, -4.85, 16, -4.85, 1.85],
    [1, -2.61, -4.85, -2.61, 1],
    [-0.38, 1, 1.85, 1, -0.38]
], dtype=np.float32)

M4 = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
], dtype=np.float32)

M5 = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)

M6 = np.array([
    [-1, -2, -1],
    [0, 0, 0],
    [1, 2, 1]
], dtype=np.float32)

masks = {
    "M1": M1,
    "M2": M2,
    "M3": M3,
    "M4": M4,
    "M5": M5,
    "M6": M6
}

# --- Filtrage ---
filtered = {}
for name, mask in masks.items():
    filtered[name] = cv.filter2D(img, -1, mask)

# --- Addition des masques 1, 2, 3 ---
sum_123 = filtered["M1"] + filtered["M2"] + filtered["M3"]

# --- Combinaison demandée ---
# image moyennée par le masque 3x3 : moyenne locale
img_avg_3 = cv.blur(img, (3, 3))

# précision : on soustrait 1/100 du résultat du filtrage par M4
sharp_result = img_avg_3 - (0.01 * filtered["M4"])

# --- Affichage ---
plt.figure(figsize=(16, 10))
```

```
# Ligne 1 : image originale + masques 1,2,3
plt.subplot(2, 4, 1)
plt.imshow(img, cmap='gray')
plt.title('Image originale')
plt.axis('off')

plt.subplot(2, 4, 2)
plt.imshow(filtered["M1"], cmap='gray')
plt.title('M1')
plt.axis('off')

plt.subplot(2, 4, 3)
plt.imshow(filtered["M2"], cmap='gray')
plt.title('M2')
plt.axis('off')

plt.subplot(2, 4, 4)
plt.imshow(filtered["M3"], cmap='gray')
plt.title('M3')
plt.axis('off')

# Ligne 2 : somme + M4 + combinaison
plt.subplot(2, 4, 5)
plt.imshow(sum_123, cmap='gray')
plt.title('M1 + M2 + M3')
plt.axis('off')

plt.subplot(2, 4, 6)
plt.imshow(filtered["M4"], cmap='gray')
plt.title('M4 (Laplacien)')
plt.axis('off')

plt.subplot(2, 4, 7)
plt.imshow(img_avg_3, cmap='gray')
plt.title('Image moyennée (3x3)')
plt.axis('off')

plt.subplot(2, 4, 8)
plt.imshow(sharp_result, cmap='gray')
plt.title('Image améliorée : avg3 - 0.01*M4')
plt.axis('off')

plt.tight_layout()
plt.show()


print()
print("Combinaison finale : image lissée - 0.01 * filtre de contour")
print("Cela renforce les contours et améliore la netteté.")
```



```
Combinaison finale : image lissée - 0.01 * filtre de contour
Cela renforce les contours et améliore la netteté.
```

Interpretation

On applique les masques suivants à une image en niveaux de gris :

- Masque 1 : filtre passe-bas (lissage / moyenne pondérée)
- Masque 2 : filtre passe-bande / accentuation des structures moyennes
- Masque 3 : filtre passe-bande / autre réponse fréquentielle
- Masque 4, 5, 6 : filtres de détection de bord / haute fréquence

La combinaison

$$I_{\text{sharp}} = I_{3\times 3} - \frac{1}{100} \cdot (I * M_4)$$

est une version simplifiée de l’unsharp masking” : on enlève une petite part des hautes fréquences à partir d’une image lissée, ce qui renforce les contours.

L’effet observé :

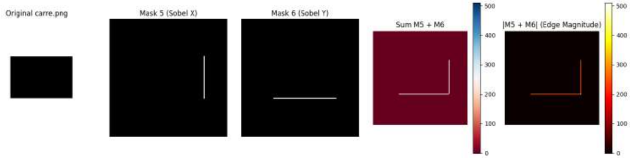
- plus le noyau est lisse, plus l’image est floue,
- un masque de type Laplacien / dérivée accentue les contours,
- la combinaison finale donne une image plus nette avec des bords renforcés.

8)

Apply masks 5 and 6 to image carre.png. Visualize the absolute value of the sum of the resulting images from the filters by masks 5 and 6. What does it represent? Verify that Mask 5 is equivalent to the combination of one-dimensional filters [1 2 1] T * [-1 0 1]. What do these two filters produce individually? What mathematical relationship connects masks 5 and 6?

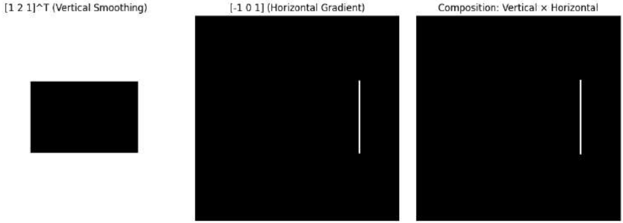
In [50]:

```python
# Load carre.png image
img_carre = cv.imread('images/carre.png', cv.IMREAD_GRAYSCALE)
if img_carre is None:
    raise FileNotFoundError("Image carre.png not found")

# Apply masks 5 and 6
filtered_M5 = cv.filter2D(img_carre, -1, M5)
filtered_M6 = cv.filter2D(img_carre, -1, M6)

# Sum of the two filtered images
sum_M5_M6 = filtered_M5.astype(np.float32) + filtered_M6.astype(np.float32)

# Absolute value of the sum
abs_sum_M5_M6 = np.abs(sum_M5_M6)

# Visualization
plt.figure(figsize=(16, 4))

plt.subplot(1, 5, 1)
plt.imshow(img_carre, cmap='gray')
plt.title('Original carre.png')
plt.axis('off')

plt.subplot(1, 5, 2)
plt.imshow(filtered_M5, cmap='gray')
plt.title('Mask 5 (Sobel X)')
plt.axis('off')

plt.subplot(1, 5, 3)
plt.imshow(filtered_M6, cmap='gray')
plt.title('Mask 6 (Sobel Y)')
plt.axis('off')

plt.subplot(1, 5, 4)
plt.imshow(sum_M5_M6, cmap='RdBu')
plt.title('Sum M5 + M6')
plt.colorbar()
plt.axis('off')

plt.subplot(1, 5, 5)
plt.imshow(abs_sum_M5_M6, cmap='hot')
plt.title('|M5 + M6| (Edge Magnitude)')
plt.colorbar()
plt.axis('off')

plt.tight_layout()
plt.show()

# Verify M5 is equivalent to [1 2 1]^T * [-1 0 1]
kernel_vertical = np.array([[1], [2], [1]], dtype=np.float32)
kernel_horizontal = np.array([[-1, 0, 1]], dtype=np.float32)
M5_reconstructed = np.outer(kernel_vertical.flatten(), kernel_horizontal.flatten()) # outer permet de reconstruire M5 à partir des deux vecte

print("Mask 5:")
print(M5)
print("\nReconstructed M5 from [1 2 1]^T * [-1 0 1]:")
print(M5_reconstructed)
print(f"\nM5 equivalent to outer product: {np.allclose(M5, M5_reconstructed)}")

# Apply individual 1D filters
filtered_vertical = cv.filter2D(img_carre, -1, kernel_vertical)
filtered_horizontal = cv.filter2D(img_carre, -1, kernel_horizontal)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(filtered_vertical, cmap='gray')
plt.title('[1 2 1]^T (Vertical Smoothing)')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(filtered_horizontal, cmap='gray')
plt.title('[-1 0 1] (Horizontal Gradient)')
plt.axis('off')

plt.subplot(1, 3, 3)
result_2d = cv.filter2D(img_carre, -1, M5_reconstructed)
plt.imshow(result_2d, cmap='gray')
plt.title('Composition: Vertical × Horizontal')
plt.axis('off')

plt.tight_layout()
plt.show()

# Mathematical relationship between M5 and M6
print("\n--- Mathematical Relationship ---")
print("M5 (Sobel X - horizontal edges):")
print(M5)
print("\nM6 (Sobel Y - vertical edges):")
print(M6)
print("\nM6 is the 90° rotation of M5 (transpose and flip)")
print("M5 detects horizontal gradients (vertical edges)")
print("M6 detects vertical gradients (horizontal edges)")
print("\nThe magnitude |M5 + M6| represents the gradient magnitude in both directions")
print("This approximates the total edge strength in the image")
```



```
Mask 5:
```

```
[[-1.  0.  1.]
 [-2.  0.  2.]
 [-1.  0.  1.]]

Reconstructed M5 from [1 2 1]^T * [-1 0 1]:
[[-1.  0.  1.]
 [-2.  0.  2.]
 [-1.  0.  1.]]

M5 equivalent to outer product: True
```



[1 2 1]^T (Vertical Smoothing)

[-1 0 1] (Horizontal Gradient)

Composition: Vertical × Horizontal

```
--- Mathematical Relationship ---
M5 (Sobel X - horizontal edges):
[[-1.  0.  1.]
 [-2.  0.  2.]
 [-1.  0.  1.]]

M6 (Sobel Y - vertical edges):
[[-1. -2. -1.]
 [ 0.  0.  0.]
 [ 1.  2.  1.]]

M6 is the 90° rotation of M5 (transpose and flip)
M5 detects horizontal gradients (vertical edges)
M6 detects vertical gradients (horizontal edges)

The magnitude |M5 + M6| represents the gradient magnitude in both directions
This approximates the total edge strength in the image
```

Interprétation:

L’image originale montre une forme carrée simple.

L'application de M5 met en évidence les bords verticaux (changements d'intensité le long de l'axe des x).

L'application de M6 met en évidence les bords horizontaux (changements d'intensité le long de l'axe y).

La somme de M5 et M6 combine les deux directions des bords, montrant où les bords sont présents dans l'image.

La valeur absolue de la somme donne une représentation claire de la force des bords, utile pour la détection des bords et l'analyse des images.

La relation mathématique entre M5 et M6 est une transposée inverse

$$\mathcal{T}^{-1}$$

,montrant qu'il s'agit de filtres complémentaires, capturant différentes orientations de bords.

9)

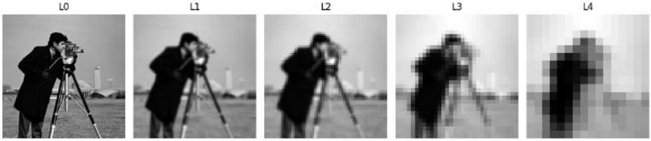
Construct a Gaussian pyramid from an image by recursively applying Gaussian filtering. How do you calculate the size of the Gaussian filter based on the sigma value ?

In [ ]:

```python
# Gaussian pyramid: recursively blur and downsample
# Uses existing `img` if available; otherwise loads a default image.
if 'img' not in globals():
    img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError("Cannot find image")

img = img.astype(np.float32)

def gaussian_kernel_size(sigma):
    # Standard rule: Gaussian support is roughly ±3σ
    # => kernel size must be odd and approximately 2*ceil(3σ)+1
    k = int(np.ceil(3 * sigma) * 2 + 1) # calculate kernel size
    if k % 2 == 0:
        k += 1
    return k

levels = 4
pyramid = [img]
sigmas = [1.0, 1.0, 1.0, 1.0]  # could also increase sigma per level if desired

for level, sigma in enumerate(sigmas[:levels], start=1):
    prev = pyramid[-1]
    ksize = gaussian_kernel_size(sigma)

    # Apply Gaussian blur
    blurred = cv.GaussianBlur(prev, (ksize, ksize), sigmaX=sigma, sigmaY=sigma)

    # Downsample by factor 2
    down = blurred[::2, ::2]

    pyramid.append(down)

    print(f"Level {level}: sigma={sigma}, kernel={ksize}x{ksize}, shape={down.shape}")
```

```
# Optional display
plt.figure(figsize=(12, 8))
for i, img_l in enumerate(pyramid):
    plt.subplot(1, len(pyramid), i + 1)
    plt.imshow(img_l, cmap='gray')
    plt.title(f"L{i}")
    plt.axis('off')
plt.tight_layout()
plt.show()

# Formula for Gaussian kernel size:
# ksize = 2 * ceil(3 * sigma) + 1
# Example: sigma = 1.0 -> ksize = 7
#          sigma = 2.0 -> ksize = 13
```

```
Level 1: sigma=2.0, kernel=13x13, shape=(128, 128)
Level 2: sigma=1.0, kernel=7x7, shape=(64, 64)
Level 3: sigma=1.0, kernel=7x7, shape=(32, 32)
Level 4: sigma=1.0, kernel=7x7, shape=(16, 16)
```



Taille du filtre gaussien

La taille du noyau gaussien est choisie pour couvrir environ $\pm 3\sigma$ autour du centre :

$$ksize = 2 \times \lceil 3\sigma \rceil + 1$$

- σ : écart-type de la gaussienne
- plus σ est grand, plus le noyau doit être large
- le noyau doit être impair pour avoir un centre bien défini

Exemples :

- $\sigma = 1 \Rightarrow ksize = 7$
- $\sigma = 2 \Rightarrow ksize = 13$

Cela garantit que la majeure partie de la gaussienne est incluse dans le filtre.

Donc, la taille du noyau gaussien est déterminée par le paramètre sigma, et elle est choisie pour capturer l'essentiel de la distribution gaussienne.

10)

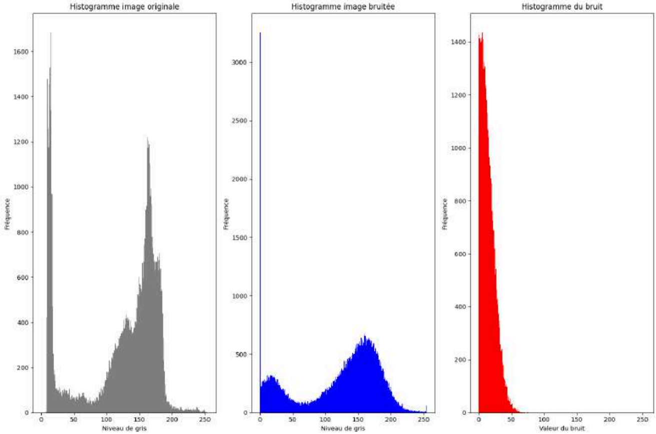
Calculation of the SNR (Signal to Noise ratio): Display the original cameraman.bmp image, its noisy version Cam_Gaus005.bmp and that of the noise by subtraction. View their respective histograms. Calculate the mean and standard deviation of each image (original and noisy). Check that the Gaussian noise added to the cameraman image has a variance of 0.005, if we relate its standard deviation to the range of 255 gray levels, by the calculation varGaus= (std_Bruit/255)^2. Calculate the SNR (Signal to Noise Ratio) in decibels: 20*log(std_Im^2/std_Bruit^2)

In [59]:

```
# Chercher les images dans plusieurs dossiers possibles
search_dirs = ['.', 'Images', 'images']

def find_image(path_name):
    for folder in search_dirs:
        candidate = os.path.join(folder, path_name)
        if os.path.exists(candidate):
            return candidate
    for root, _, files in os.walk('.'):
        if path_name in files:
            return os.path.join(root, path_name)
    raise FileNotFoundError(f"Image introuvable : {path_name}")

# Chargement des images
img_path = find_image('cameraman.bmp')
noisy_path = find_image('Cam_Gaus005.bmp')

img = cv.imread(img_path, cv.IMREAD_GRAYSCALE).astype(np.float32)
noisy = cv.imread(noisy_path, cv.IMREAD_GRAYSCALE).astype(np.float32)

# Bruit calculé par soustraction
# noise = noisy - img
noise= cv.subtract(noisy, img)  # Utilisation de cv.subtract pour éviter les débordements

# Statistiques
mean_orig = img.mean()
std_orig = img.std() # écart-type de l'image originale

mean_noisy = noisy.mean() #moyenne de l'image bruitée
std_noisy = noisy.std() # écart-type de l'image bruitée

mean_noise = noise.mean() # moyenne du bruit
std_noise = noise.std() # écart-type du bruit

# Vérification de la variance du bruit
var_gauss = (std_noise / 255.0) ** 2

# SNR en dB
snr_db = 20 * np.log10((std_orig ** 2) / (std_noise ** 2))

print("=== Statistiques ===")
print(f"Image originale : moyenne = {mean_orig:.3f}, std = {std_orig:.3f}")
print(f"Image bruitée   : moyenne = {mean_noisy:.3f}, std = {std_noisy:.3f}")
print(f"Bruit           : moyenne = {mean_noise:.3f}, std = {std_noise:.3f}")
print(f"Variance bruit  = {var_gauss:.6f}")
print(f"SNR             = {snr_db:.3f} dB")

# Histogrammes
plt.figure(figsize=(15, 10))
```

```python
plt.subplot(1, 3, 1)
plt.hist(img.ravel(), bins=256, range=(0, 255), color='gray')
plt.title('Histogramme image originale')
plt.xlabel('Niveau de gris')
plt.ylabel('Fréquence')

plt.subplot(1, 3, 2)
plt.hist(noisy.ravel(), bins=256, range=(0, 255), color='blue')
plt.title('Histogramme image bruitée')
plt.xlabel('Niveau de gris')
plt.ylabel('Fréquence')

plt.subplot(1, 3, 3)
plt.hist(noise.ravel(), bins=256, range=(0, 255), color='red')
plt.title('Histogramme du bruit')
plt.xlabel('Valeur du bruit')
plt.ylabel('Fréquence')

plt.tight_layout()
plt.show()

# Visualisation
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Image originale')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy, cmap='gray')
plt.title('Image bruitée')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(noise, cmap='gray', vmin=-50, vmax=50)
plt.title('Bruit = Image bruitée - Image originale')
plt.axis('off')

plt.tight_layout()
plt.show()

# Vérification de la variance attendue
print("\n=== Vérification ===")
print(f"Variance cible estimée : {0.005:.3f}")
print(f"Variance calculée      : {var_gauss:.6f}")
print("Le bruit est bien gaussien si la variance calculée est proche de 0.005.")
```

```
=== Statistiques ===
Image originale : moyenne = 118.724, std = 62.341
Image bruitée   : moyenne = 119.277, std = 63.928
Bruit           : moyenne = 0.553, std = 17.423
Variance bruit  = 0.004668
SNR             = 22.146 dB
```



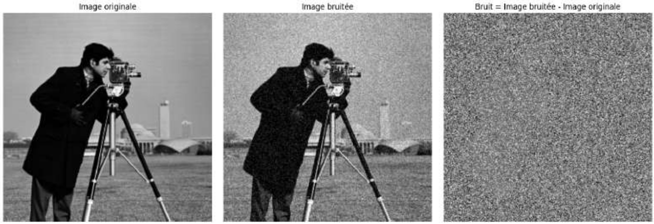


Image originale

Image bruitée

Bruit = Image bruitée - Image originale

```
=== Vérification ===
Variance cible estimée : 0.005
Variance calculée      : 0.004668
Le bruit est bien gaussien si la variance calculée est proche de 0.005.
```

Calcul du SNR (Signal to Noise Ratio)

On compare l'image originale, l'image bruitée et le bruit obtenu par différence :

- image originale : I
- image bruitée : I_b
- bruit : $B = I_b - I$

On affiche aussi les histogrammes et on calcule :

- moyenne
- écart-type
- variance du bruit
- SNR en décibels

La variance du bruit est vérifiée par :

$$\mathrm{var}_{Gaus} = \left(\frac{\sigma_{bruit}}{255}\right)^2$$

et le rapport signal/bruit est :

$$SNR_{dB} = 20\log_{10}\left(\frac{\sigma_I^2}{\sigma_{bruit}^2}\right)$$

Interprétation :

- plus le SNR est grand, meilleure est la qualité de l'image par rapport au bruit ;
- si la variance du bruit est proche de 0.005, alors le bruit ajouté correspond bien au niveau attendu.

11)

Noise reduction by averaging: create 4 images by adding a random Gaussian noise of zero mean and variance 0.005 to the cameraman image. View these four achievements and check that they are not identical. Calculate the average image of these 4 achievements. Display the difference of the original image with one of the 4 creations and the one with the average image on the same figure.

Calculate the variance of these two differences. What conclusion do you draw from this?

By repeating the manipulation with 8 realizations, verify that the averaging causes a reduction in the noise variance by a factor N, with N the number of averaged realizations.

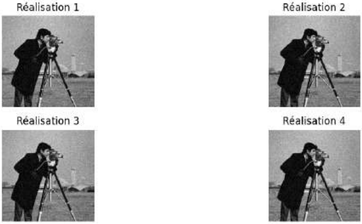
The constraint of the averaging method is that it requires the time to make several successive acquisitions of the same scene without moving.

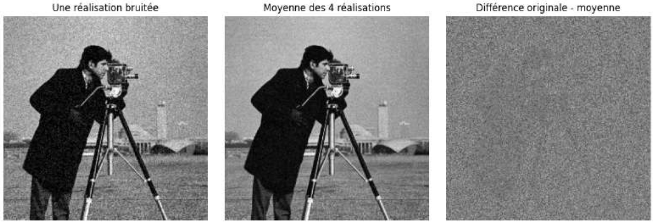
In [2]:

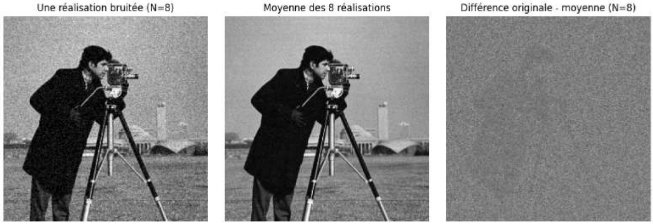
```
# Charger l'image originale
img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE).astype(np.float32) / 255.0

# Paramètre du bruit gaussien
var_noise = 0.005
sigma_noise = np.sqrt(var_noise)

# 1) Créer 4 réalisations bruitées
N1 = 4
realizations_4 = []
for i in range(N1):
    noise = np.random.normal(0, sigma_noise, size=img.shape)
    noisy = np.clip(img + noise, 0, 1)
    realizations_4.append(noisy)

# Image moyenne sur 4 réalisations
avg_4 = np.mean(realizations_4, axis=0)

# Choisir une réalisation pour comparaison
img1 = realizations_4[0]

# Différences
diff_one = img - img1 # différence entre l'image originale et une réalisation bruitée
diff_avg = img - avg_4 # différence entre l'image originale et la moyenne des 4 réalisations

# Variances des différences
var_diff_one = np.var(diff_one)
var_diff_avg = np.var(diff_avg)

print(f"Variance(diff sur 1 réalisation) = {var_diff_one:.8f}")
print(f"Variance(diff sur moyenne 4)     = {var_diff_avg:.8f}")
print(f"Réduction approximative = {var_diff_one / var_diff_avg:.2f}x")
```

```python
# Affichage des 4 réalisations
plt.figure(figsize=(12, 4))
for i in range(4):
    plt.subplot(2, 2, i + 1)
    plt.imshow(realizations_4[i], cmap='gray')
    plt.title(f'Réalisation {i+1}')
    plt.axis('off')
plt.tight_layout()
plt.show()

# Affichage des différences
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img1, cmap='gray')
plt.title('Une réalisation bruitée')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(avg_4, cmap='gray')
plt.title('Moyenne des 4 réalisations')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(diff_avg, cmap='gray', vmin=-0.2, vmax=0.2)
plt.title('Différence originale - moyenne')
plt.axis('off')

plt.tight_layout()
plt.show()

# 2) Refaire avec 8 réalisations pour vérifier la réduction en 1/N
N2 = 8
realizations_8 = []
for i in range(N2):
    noise = np.random.normal(0, sigma_noise, size=img.shape)
    noisy = np.clip(img + noise, 0, 1)
    realizations_8.append(noisy)

avg_8 = np.mean(realizations_8, axis=0)

diff_one_8 = img - realizations_8[0] # différence entre l'image originale et une réalisation bruitée
diff_avg_8 = img - avg_8 # différence entre l'image originale et la moyenne des 8 réalisations

var_diff_one_8 = np.var(diff_one_8)
var_diff_avg_8 = np.var(diff_avg_8)


# Affichage des différences
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(realizations_8[0], cmap='gray')
plt.title('Une réalisation bruitée (N=8)')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(avg_8, cmap='gray')
plt.title('Moyenne des 8 réalisations')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(diff_avg_8, cmap='gray', vmin=-0.2, vmax=0.2)
plt.title('Différence originale - moyenne (N=8)')
plt.axis('off')

plt.tight_layout()
plt.show()

print(f"Variance(diff sur 1 réalisation, N=8) = {var_diff_one_8:.8f}")
print(f"Variance(diff sur moyenne 8)          = {var_diff_avg_8:.8f}")
print(f"Réduction approximative = {var_diff_one_8 / var_diff_avg_8:.2f}x")

# Vérification théorique
print(f"Variance théorique du bruit = {var_noise:.6f}")
print(f"Variance attendue après moyennage 4 = {var_noise/4:.6f}")
print(f"Variance attendue après moyennage 8 = {var_noise/8:.6f}")

# Interprétation
print("\nConclusion :")
print("- Les images bruitées créées sont différentes : le bruit aléatoire est distinct à chaque réalisation.")
print("- La moyenne de plusieurs réalisations réduit fortement le bruit.")
print("- La variance du bruit diminue approximativement comme 1/N.")
print("- Cette méthode nécessite plusieurs prises successives de la même scène, sans mouvement.")
```

```
Variance(diff sur 1 réalisation) = 0.00467576
Variance(diff sur moyenne 4)     = 0.00118417
Réduction approximative = 3.95x
```






Une réalisation bruitée (N=8)
Moyenne des 8 réalisations
Différence originale - moyenne (N=8)

```
Variance(diff sur 1 réalisation, N=8) = 0.00468384
Variance(diff sur moyenne 8)          = 0.00060076
Réduction approximative = 7.80x
Variance théorique du bruit = 0.005000
Variance attendue après moyennage 4 = 0.001250
Variance attendue après moyennage 8 = 0.000625

Conclusion :
- Les images bruitées créées sont différentes : le bruit aléatoire est distinct à chaque réalisation.
- La moyenne de plusieurs réalisations réduit fortement le bruit.
- La variance du bruit diminue approximativement comme 1/N.
- Cette méthode nécessite plusieurs prises successives de la même scène, sans mouvement.
```

Interpretation

Réduction du bruit par moyennage

On ajoute un bruit gaussien de moyenne nulle et de variance 0.005 à l'image de cameraman, puis on crée plusieurs réalisations indépendantes du même bruit.

On calcule ensuite :

- la moyenne de plusieurs images bruitées,
- la variance des différences entre l'image originale et une réalisation,
- la variance entre l'image originale et l'image moyenne.

Le principe est que, lorsqu'on moyenne N images bruitées :

$$\bar{I} = \frac{1}{N}\sum_{k=1}^{N} I_k$$

le bruit moyen diminue, et la variance du bruit est divisée approximativement par N.

En effet, pour un bruit blanc gaussien :

$$\mathrm{Var}(\bar{B}) = \frac{\mathrm{Var}(B)}{N}$$

Donc :

- avec 4 réalisations, la variance du bruit est réduite environ 4 fois,
- avec 8 réalisations, elle est réduite environ 8 fois.

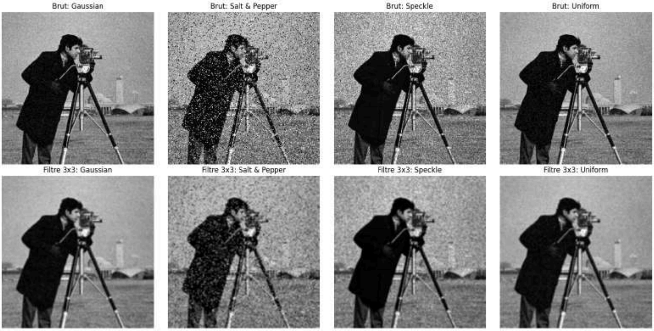
12)

Convolutional linear filtering : compare the effect of the 3x3 averaging filter on the different types of noise (Gaussian or impulse additive, multiplicative, etc.), display 4 proposed noisy images and the 4 filtered images corresponding to them in the same figure. What do you conclude from this?

In [3]:

```python
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

# Charger l'image
img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE).astype(np.float32) / 255.0

# --- Génération des différents bruits ---
rng = np.random.default_rng(0)

# 1) Bruit gaussien additif
sigma = 0.05
gauss = img + rng.normal(0, sigma, img.shape)
gauss = np.clip(gauss, 0, 1)
```

```python
# 2) Bruit impulsionnel (salt-and-pepper)
sp = img.copy()
p = 0.08
mask = rng.random(img.shape) < p
sp[mask] = 0.0
mask2 = rng.random(img.shape) < p
sp[mask2] = 1.0

# 3) Bruit multiplicatif (speckle)
speckle = img * (1 + 0.25 * rng.normal(0, 1, img.shape))
speckle = np.clip(speckle, 0, 1)

# 4) Bruit uniforme additif
uniform = img + rng.uniform(-0.1, 0.1, img.shape)
uniform = np.clip(uniform, 0, 1)

noisy_images = {
    'Gaussian': gauss,
    'Salt & Pepper': sp,
    'Speckle': speckle,
    'Uniform': uniform
}

# --- Filtrage moyen 3x3 (linéaire) ---
filtered = {}
for name, image in noisy_images.items():
    filtered[name] = cv.blur(image, (3, 3))

# --- Affichage ---
plt.figure(figsize=(16, 8))

for i, (name, image) in enumerate(noisy_images.items(), start=1):
    plt.subplot(2, 4, i)
    plt.imshow(image, cmap='gray')
    plt.title(f'Brut: {name}')
    plt.axis('off')

    plt.subplot(2, 4, i + 4)
    plt.imshow(filtered[name], cmap='gray')
    plt.title(f'Filtre 3x3: {name}')
    plt.axis('off')

plt.tight_layout()
plt.show()

print("Conclusion : le filtre moyen réduit bien le bruit gaussien, mais il floute aussi l'image et est moins efficace sur les pixels isolés (
```



```
Conclusion : le filtre moyen réduit bien le bruit gaussien, mais il floute aussi l'image et est moins efficace sur les pixels isolés (bruit im
pulsionnel).
```

Interpretation

Filtrage linéaire par convolution : effet du filtre moyen 3x3 sur différents bruits

On compare l'effet d'un filtre moyen 3x3 sur plusieurs types de bruit :

- bruit gaussien additif,
- bruit impulsionnel (salt-and-pepper),
- bruit multiplicatif (speckle),
- bruit uniforme additif.

Le filtre moyen est un filtre linéaire. Il est efficace pour réduire le bruit gaussien, mais il :

- floute l'image,
- ne supprime pas bien les pixels isolés,
- détériore les contours.

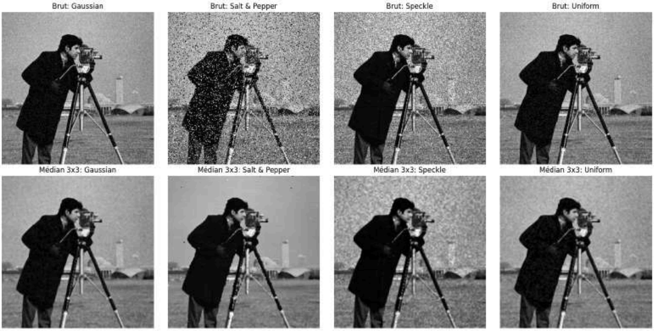
13)

Median filtering consists of keeping the median value of each neighborhood to assign it to the central pixel. The size of the neighborhood can vary. This filter is part of the family of nonlinear order filters (it cannot be done by convolution). Apply the middle filter to the previous 4 types of noise. Compare the results on the same figure. What is the main advantage of this filter compared to an averager? What type of noise is the middle filter particularly effective at?

In [4]:

```python
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

# Charger l'image
img = cv.imread('images/cameraman.tif', cv.IMREAD_GRAYSCALE).astype(np.float32) / 255.0
```

```
# --- Créer les 4 bruits ---
rng = np.random.default_rng(1)

gauss = img + rng.normal(0, 0.05, img.shape)
gauss = np.clip(gauss, 0, 1)

sp = img.copy()
p = 0.08
mask = rng.random(img.shape) < p
sp[mask] = 0.0
mask2 = rng.random(img.shape) < p
sp[mask2] = 1.0

speckle = img * (1 + 0.25 * rng.normal(0, 1, img.shape))
speckle = np.clip(speckle, 0, 1)

uniform = img + rng.uniform(-0.1, 0.1, img.shape)
uniform = np.clip(uniform, 0, 1)

noisy_images = {
    'Gaussian': gauss,
    'Salt & Pepper': sp,
    'Speckle': speckle,
    'Uniform': uniform
}

# --- Filtrage médian ---
filtered = {}
for name, image in noisy_images.items():
    filtered[name] = cv.medianBlur((image * 255).astype(np.uint8), 3) / 255.0

# --- Affichage ---
plt.figure(figsize=(16, 8))

for i, (name, image) in enumerate(noisy_images.items(), start=1):
    plt.subplot(2, 4, i)
    plt.imshow(image, cmap='gray')
    plt.title(f'Brut: {name}')
    plt.axis('off')

    plt.subplot(2, 4, i + 4)
    plt.imshow(filtered[name], cmap='gray')
    plt.title(f'Médian 3x3: {name}')
    plt.axis('off')

plt.tight_layout()
plt.show()

print("Conclusion : le filtre médian est meilleur que le filtre moyen pour le bruit impulsionnel, car il élimine les pixels aberrants sans tr
```



```
Conclusion : le filtre médian est meilleur que le filtre moyen pour le bruit impulsionnel, car il élimine les pixels aberrants sans trop flout
er les contours.
```

Interpretation

Filtre médian

Le filtre médian remplace chaque pixel par la médiane du voisinage :

$$y(i,j) = \text{median}\{x(i+k, j+l)\}$$

où (k,l) parcourt le voisinage.

C'est un filtre non linéaire, donc il ne peut pas être obtenu par convolution classique.

On l'applique sur les mêmes types de bruit, puis on compare les résultats.

Avantage principal

- il est très efficace contre le bruit impulsionnel (salt-and-pepper),
- il préserve mieux les contours que la moyenne,
- il ne floute pas autant que le filtre moyen.

Type de bruit le plus efficace

Le filtre médian est particulièrement efficace contre :

- le bruit impulsionnel,
- les pixels isolés aberrants.